AG TECHNOLOGIES

Architecture de Services Partagés

Cahier des Charges Technique

Service Wallet Centralisé

Wallet Service — AGT-Wallet

Version	1.1
Date	Avril 2026
Statut	Implementation-ready
Classification	Confidentiel
Auteur	Équipe Architecture
Dépendances	Auth v2.1, Users v1.0, Paiement v1.2, Notification v1.2

Document confidentiel — Réservé à l'usage interne d'AG Technologies

Historique des Révisions

Ver.	Date	Auteur	Description
1.0	2026-04-04	Équipe Archi.	Création initiale. Wallet unique cross-plateforme. Ledger double-entry obligatoire. Comptes système (user, organization, platform, escrow, external). Cash-in via Paiement, cash-out vers Mobile Money/banque. Virements user-to-user. Holds/réservations. Partages automatisés (commissions). Idempotence. Events RabbitMQ. Diagramme PlantUML en annexe.
1.1	2026-04-04	Équipe Archi.	Version finale : statut *Implementation-ready*. Encart principes fondamentaux ajouté (lisibilité). Schéma visuel des flux financiers. `ledger_reference_id` ajouté sur `ledger_transactions`. Health check version alignée.

Table des matières

1 Introduction **4**
1.1 Périmètre . . . 4
1.2 Hors périmètre . . . 4

2 Contexte & Problématique **5**
2.1 Contexte . . . 5
2.2 Problématique . . . 5

3 Vision & Objectifs **5**
3.1 Vision . . . 5
3.2 Architecture des comptes . . . 5
3.3 Objectifs . . . 6

4 Parties Prenantes & Utilisateurs **6**

5 Besoins Fonctionnels **6**
5.1 Gestion des Comptes . . . 6
5.2 Ledger & Écritures . . . 7
5.3 Cash-in (Entrée de fonds) . . . 7
5.4 Cash-out (Sortie de fonds) . . . 8
5.5 Holds / Réservation de fonds . . . 8
5.6 Virements & Partages . . . 9
5.7 Administration . . . 9

6 Besoins Non-Fonctionnels **10**

7 Architecture Technique **10**
7.1 Vue d'ensemble . . . 10
7.2 Stack Technique . . . 11
7.3 Dépendances & Règles d'Architecture . . . 11
7.4 Stratégie d'intégrité . . . 11
7.5 Diagramme d'architecture . . . 11
7.6 Flux financiers (v1.1) . . . 12

8 Modélisation **12**
8.1 Diagramme de Classes . . . 12
8.2 MLD — Modèle Logique de Données . . . 13
8.2.1 Table `accounts` . . . 13
8.2.2 Table `ledger_transactions` . . . 14
8.2.3 Table `ledger_entries` . . . 14
8.2.4 Table `holds` . . . 15
8.2.5 Table `cashout_requests` . . . 15
8.2.6 Table `split_rules` . . . 16

9 Contrats d'API **16**
9.1 Health Check . . . 16
9.1.1 GET /wallet/health . . . 16
9.2 Comptes . . . 16
9.2.1 GET /wallet/accounts/me . . . 16
9.2.2 GET /wallet/accounts/{accountId} . . . 17
9.2.3 POST /wallet/accounts/{accountId}/freeze . . . 17

9.2.4 POST /wallet/accounts/{accountId}/unfreeze . . . 17
9.3 Opérations Ledger . . . 17
9.3.1 POST /wallet/credit . . . 17
9.3.2 POST /wallet/debit . . . 17
9.3.3 POST /wallet/transfer . . . 18
9.3.4 POST /wallet/split . . . 18
9.3.5 GET /wallet/accounts/{accountId}/transactions . . . 19
9.4 Holds . . . 19
9.4.1 POST /wallet/holds . . . 19
9.4.2 POST /wallet/holds/{holdId}/capture . . . 19
9.4.3 POST /wallet/holds/{holdId}/release . . . 19
9.5 Cash-out . . . 19
9.5.1 POST /wallet/cashout . . . 19
9.5.2 GET /wallet/cashout/{cashoutId} . . . 20
9.5.3 GET /wallet/cashout . . . 20
9.6 Split Rules . . . 20
9.6.1 POST /wallet/split-rules . . . 20
9.6.2 GET /wallet/split-rules . . . 20
9.6.3 PUT /wallet/split-rules/{ruleId} . . . 20
9.7 Administration . . . 20
9.7.1 GET /wallet/admin/stats . . . 20
9.7.2 POST /wallet/admin/audit-ledger . . . 21
9.7.3 POST /wallet/admin/adjustment . . . 21
9.7.4 DELETE /wallet/by-user/{userId} . . . 21

10 Contrats Inter-Services **21**
10.1 Wallet → Auth (JWT / S2S) . . . 21
10.2 Paiement → Wallet (événements RabbitMQ entrants) . . . 22
10.3 Wallet → Paiement (cash-out) . . . 22
10.4 Wallet → Notification (alertes) . . . 22
10.5 Wallet — Événements sortants génériques . . . 22
10.6 Users → Wallet (Purge RGPD) . . . 23

11 Matrice d'Autorisation **23**

12 Recommandations & Évolutions **23**

1 Introduction

Ce document constitue le cahier des charges du **Service Wallet Centralisé** (ci-après « Wallet Service ») destiné à l'écosystème AGT. Il sert de référence unique pour la conception, le développement et les décisions techniques.

Principes fondamentaux — À lire en premier

1. **Double-entry ledger** : chaque mouvement = 1 débit + 1 crédit équilibrés. Pas d'exception.
2. **Append-only** : aucun UPDATE, aucun DELETE sur les écritures. Corrections par écriture inverse (reversal).
3. **Ledger = source de vérité** : le champ `balance` est un cache. Le solde réel = $\sum$ crédits − $\sum$ débits.
4. **Transactions atomiques** : toute opération d'écriture est encapsulée dans une transaction PostgreSQL (SERIALIZABLE).
5. **Idempotence** : chaque mouvement porte un `idempotency_key` unique. Doublon = retour du résultat existant.
6. **Holds obligatoires** : tout cash-out ou paiement en cours passe par une réservation de fonds avant exécution.

1.1 Périmètre

Le service Wallet gère la **comptabilité interne** de l'écosystème AGT : comptes virtuels, mouvements financiers (crédits, débits, virements, commissions), réservations de fonds (holds), cash-out vers providers externes, et traçabilité complète via un ledger double-entry.

Principe fondamental

Le Wallet est un **ledger comptable interne**. Chaque mouvement d'argent est enregistré comme une écriture double (débit + crédit). Le solde n'est **jamais** un simple champ incrémenté : il est **dérivé** de la somme des écritures du ledger ou maintenu de manière cohérente via des transactions atomiques.

1.2 Hors périmètre

- Authentification (déléguée au Service Auth v2.1)
- Profils utilisateur (Service Users v1.0)
- Exécution des paiements externes (Service Paiement v1.2)
- Gestion des abonnements (Service Abonnement v1.2)
- Logique métier propre aux plateformes (commissions, règles de partage)
- Envoi de notifications (Service Notification v1.2)
- Comptabilité générale / ERP (hors périmètre écosystème)

Convention d'identité

Dans tout ce document :

- `user_id` désigne `users_auth.id` (= `sub` du JWT)
- `platform_id` désigne un UUID du registre Auth (`platforms.id`)
- Le wallet est **unique par user** pour tout l'écosystème, mais chaque écriture porte un `platform_id` pour traçabilité

2 Contexte & Problématique

2.1 Contexte

Plateforme	Statut	Besoin wallet
AGT-Bot	En développement	Crédits bonus, remboursements futurs
AGT-Market	En développement	Encaissement vendeurs, commissions, paiement fournisseurs/livreurs
MboaMove	En développement	Gains chauffeurs, commissions AGT, cash-out vers Mobile Money
SALMA	Migration prévue	Crédits formation, remboursements
Futures	Planifié	Tout flux financier interne à l'écosystème

2.2 Problématique

Sans ledger centralisé, chaque plateforme gérerait ses propres soldes avec un simple champ `balance`, entraînant : incohérences comptables, impossibilité d'auditer les mouvements, race conditions sur les soldes, absence de réservation de fonds, et aucune vision consolidée des flux financiers de l'écosystème.

3 Vision & Objectifs

3.1 Vision

Un **ledger double-entry centralisé** qui enregistre chaque mouvement financier de l'écosystème. Le wallet d'un utilisateur est unique et cross-plateforme : ses gains AGT-Market et MboaMove arrivent dans le même solde.

Séparation des responsabilités financières (rappel)

- **Abonnement** décide quoi facturer
- **Paiement** exécute la transaction externe
- **Wallet** comptabilise le mouvement interne et répartit les fonds

Le Wallet ne parle **jamais** directement aux providers de paiement. Il passe toujours par le Service Paiement pour les cash-in et cash-out.

3.2 Architecture des comptes

Type de compte	Code	Rôle
Wallet utilisateur	`user`	Solde de l'utilisateur (cross-plateforme)
Wallet organisation	`organization`	Solde d'une organisation (B2B)
Compte plateforme	`platform_system`	Commissions et frais prélevés par AGT
Compte de transit	`escrow`	Fonds en attente de confirmation (hold)
Compte externe	`external`	Représente l'entrée/sortie vers providers (Orange Money, Stripe, etc.)

Exemple : achat sur AGT-Market (10 000 XAF)

Étape 1 — Cash-in (paiement confirmé) :
- DEBIT `external(orange_money)` 10 000 XAF

— CREDIT `escrow(market)` 10 000 XAF
Étape 2 — Livraison confirmée (libération escrow) :
— DEBIT `escrow(market)` 10 000 XAF
— CREDIT `wallet_vendeur` 8 500 XAF (85%)
— CREDIT `platform_system(agt)` 1 500 XAF (15% commission)
Règle comptable : la somme des débits = la somme des crédits, toujours.

3.3 Objectifs

#	Objectif	Indicateur
O1	Ledger double-entry	Chaque mouvement = 1 débit + 1 crédit (ou N débits + N crédits équilibrés)
O2	Wallet unique	Un seul wallet par user, cross-plateforme
O3	Traçabilité totale	Chaque écriture porte source, plateforme, référence, timestamp
O4	Holds / escrow	Réservation de fonds atomique avec TTL
O5	Cash-out MVP	Retrait vers Mobile Money / banque via Paiement
O6	Virements	User→user, plateforme→user, partages automatisés
O7	Idempotence	Chaque mouvement porte un `idempotency_key` unique
O8	Solde cohérent	Solde = somme des écritures, vérifiable à tout moment

4 Parties Prenantes & Utilisateurs

Rôle	Type	Responsabilité
Product Owner	Interne	Priorisation des fonctionnalités
Équipe Backend	Interne	Développement et maintenance
Équipe Frontend	Interne	UI solde, historique, cash-out
DevOps	Interne	Infrastructure, monitoring, backups
Admin global AGT	Interne	Réconciliation, audit, comptes système
Admin plateforme	Interne	Consultation mouvements de sa plateforme
Services internes	Interne	Paiement (cash-in/out), plateformes (partages)
Utilisateurs finaux	Externe	Consultation solde, historique, retrait

5 Besoins Fonctionnels

5.1 Gestion des Comptes

ID	Fonctionnalité	Description	Priorité
F-ACC-01	Création wallet user	Créer automatiquement un wallet à la première interaction financière	**Urgent**
F-ACC-02	Création wallet org	Créer un wallet pour une organisation (lié au subscriber B2B Abonnement)	**Urgent**
F-ACC-03	Comptes système	Provisionner les comptes platform_system, escrow, external au déploiement	**Urgent**
F-ACC-04	Consultation solde	Retourner le solde disponible (= solde total − holds en cours)	**Urgent**
F-ACC-05	Gel de compte	Admin peut geler un wallet (aucun mouvement autorisé)	Normale
F-ACC-06	Statut wallet	active, frozen, closed	**Urgent**

5.2 Ledger & Écritures

ID	Fonctionnalité	Description	Priorité
F-LED-01	Écriture double-entry	Chaque mouvement génère au minimum 1 ligne débit + 1 ligne crédit équilibrées	**Urgent**
F-LED-02	Écriture multi-leg	Support des écritures à N lignes (ex : 1 débit, 2 crédits pour partage commission)	**Urgent**
F-LED-03	Immutabilité	Aucune écriture ne peut être modifiée ou supprimée. Correction = écriture inverse	**Urgent**
F-LED-04	Idempotence	Chaque opération porte un `idempotency_key`. Doublon = retour du résultat existant	**Urgent**
F-LED-05	Référence croisée	Chaque écriture porte la référence du déclencheur (payment_id, order_id, etc.)	**Urgent**
F-LED-06	Historique	Consultation des écritures avec filtres (date, type, plateforme, montant)	**Urgent**
F-LED-07	Vérification d'intégrité	Endpoint admin : vérifier que $\sum$ débits = $\sum$ crédits sur tout le ledger	Normale

Règle comptable fondamentale

Invariant : pour chaque transaction dans le ledger, la somme des montants débités doit être **strictement égale** à la somme des montants crédités. Toute tentative de créer une écriture déséquilibrée est rejetée avec une erreur 400.
Solde d'un compte : $solde = \sum crdits - \sum dbits$ sur toutes les lignes du ledger pour ce compte.
Solde disponible : $disponible = solde - \sum holdspending$.
Contrôle de solde : aucun débit n'est autorisé si le solde disponible est insuffisant (sauf pour les comptes système `external` qui peuvent avoir un solde négatif par nature).

5.3 Cash-in (Entrée de fonds)

ID	Fonctionnalité	Description	Priorité
F-CIN-01	Crédit sur payment.confirmed	À la réception de l'événement Paiement, créditer le wallet concerné	**Urgent**
F-CIN-02	Crédit en escrow	Si le paiement est lié à une commande, créditer l'escrow (pas le wallet final)	**Urgent**
F-CIN-03	Crédit direct	Crédit admin (remboursement, ajustement, bonus)	Normale

5.4 Cash-out (Sortie de fonds)

ID	Fonctionnalité	Description	Priorité
F-COUT-01	Demande de retrait	L'utilisateur demande un cash-out vers Mobile Money ou compte bancaire	**Urgent**
F-COUT-02	Vérification solde	Vérifier solde disponible $\geq$ montant demandé	**Urgent**
F-COUT-03	Hold préalable	Réserver le montant (hold) avant d'envoyer l'ordre à Paiement	**Urgent**
F-COUT-04	Envoi vers Paiement	Émettre un ordre de paiement sortant via Paiement (event ou API)	**Urgent**
F-COUT-05	Confirmation cash-out	Sur `payment.confirmed`, libérer le hold et débiter définitivement	**Urgent**
F-COUT-06	Échec cash-out	Sur `payment.failed`, libérer le hold (fonds restituables)	**Urgent**
F-COUT-07	Seuils de retrait	Montant minimum, plafond journalier, configurable par plateforme	Normale

Flux cash-out complet

1. User demande retrait de 5 000 XAF vers Orange Money
2. Wallet vérifie solde disponible $\geq$ 5 000
3. Wallet crée un hold de 5 000 XAF (DEBIT `wallet_user` $\rightarrow$ CREDIT `escrow(cashout)`)
4. Wallet émet `wallet.cashout.requested` vers Paiement
5. Paiement exécute le transfert Orange Money
6. Paiement émet `payment.confirmed`
7. Wallet libère l'escrow (DEBIT `escrow(cashout)` $\rightarrow$ CREDIT `external(orange_money)`)
8. Wallet émet `wallet.cashout.completed`

Si échec à l'étape 6 : Wallet annule le hold (DEBIT `escrow(cashout)` $\rightarrow$ CREDIT `wallet_user`).

5.5 Holds / Réservation de fonds

ID	Fonctionnalité	Description	Priorité
F-HLD-01	Création hold	Réserver un montant sur un wallet (solde disponible diminué)	**Urgent**
F-HLD-02	Capture hold	Confirmer le hold (débit définitif)	**Urgent**
F-HLD-03	Release hold	Annuler le hold (fonds libérés)	**Urgent**
F-HLD-04	Expiration hold	Cron : libérer automatiquement les holds expirés (TTL configurable)	**Urgent**
F-HLD-05	Hold partiel	Capturer partiellement un hold (ex : commande modifiée)	Normale

5.6 Virements & Partages

ID	Fonctionnalité	Description	Priorité
F-VIR-01	Virement user→user	Transfert entre deux wallets utilisateurs	**Urgent**
F-VIR-02	Virement plateforme→user	Crédit d'un wallet par un compte système	**Urgent**
F-VIR-03	Partage automatisé	Répartition d'un montant selon des règles de split (commissions)	**Urgent**
F-VIR-04	Règles de split	La plateforme définit les règles (ex : 85% vendeur, 15% AGT) via l'API	**Urgent**
F-VIR-05	Partage multi-bénéficiaires	Split vers N wallets en une seule transaction (vendeur + livreur + AGT)	**Urgent**

Exemple : partage MboaMove

Course à 3 000 XAF. Règle de split : 80% chauffeur, 20% AGT.
Écriture atomique (1 transaction, 3 lignes) :

- DEBIT `escrow(mboamove)` 3 000 XAF
- CREDIT `wallet_chauffeur` 2 400 XAF
- CREDIT `platform_system(agt)` 600 XAF

5.7 Administration

ID	Fonctionnalité	Description	Priorité
F-ADM-01	Stats globales	Solde total écosystème, volume mouvements, par plateforme	Normale
F-ADM-02	Audit ledger	Vérifier l'équilibre global $\sum$ débits = $\sum$ crédits	**Urgent**
F-ADM-03	Ajustement admin	Crédit ou débit correctif avec justification obligatoire	**Urgent**
F-ADM-04	Gel / dégel wallet	Geler un wallet suspect, dégeler après vérification	**Urgent**
F-ADM-05	Export écritures	Export CSV/JSON des écritures par période et par compte	Normale

6 Besoins Non-Fonctionnels

ID	Catégorie	Exigence	Cible
NF-01	Performance	Écriture ledger	< 100ms (P95)
NF-02	Performance	Consultation solde	< 50ms (P95)
NF-03	Performance	Throughput écritures	500 tx/s min
NF-04	Disponibilité	Uptime	99.9%
NF-05	Sécurité	Auth obligatoire	JWT ou S2S
NF-06	Sécurité	Rate limiting	Configurable
NF-07	Sécurité	Aucune suppression physique	Append-only ledger
NF-08	Intégrité	Transactions atomiques	PostgreSQL SERIALIZABLE ou SELECT FOR UPDATE
NF-09	Intégrité	Vérification d'équilibre périodique	Cron quotidien
NF-10	Scalabilité	Horizontal	Stateless, multi-instance
NF-11	Observabilité	Logs structurés	JSON, request ID, masquage montants sensibles
NF-12	Observabilité	Health check	`GET /wallet/health`
NF-13	Données	Timestamps	TIMESTAMPTZ
NF-14	Traitement	Crons holds expirés	Celery Beat, retry 3x backoff, DLQ

7 Architecture Technique

7.1 Vue d'ensemble

Le service Wallet est un microservice REST stateless, déployé en conteneur Docker. Il expose une API JSON et consomme/produit des événements RabbitMQ. C'est le service le plus critique en termes d'intégrité de données de tout l'écosystème.

7.2 Stack Technique

Composant	Technologie
Langage & Framework	Python 3.11+ / Django 5.x
API	Django REST Framework (DRF)
Base de données	PostgreSQL 15+ (niveau d'isolation SERIALIZABLE pour les écritures ledger)
ORM	Django ORM
Migrations	Django Migrations
Cache	Redis 7+ (soldes cachés, idempotence)
Scheduler	Celery + Celery Beat (holds expirés, audit quotidien)
Message Broker	RabbitMQ (événements entrants/sortants)
Validation	DRF Serializers
Conteneurisation	Docker + Docker Compose
CI/CD	GitHub Actions

7.3 Dépendances & Règles d'Architecture

Service	Type	Usage
Service Auth	Interne	Validation JWT / S2S
Service Users	Interne	Résolution `user_id`, vérification statut actif
Service Paiement	Interne	Cash-in (`payment.confirmed` → crédit wallet). Cash-out (Wallet émet ordre → Paiement exécute).
Service Notification	Interne	Alertes : crédit reçu, cash-out confirmé, solde faible
Plateformes	Interne	Définition des règles de split, déclenchement des partages

Règle stricte : Wallet ne touche jamais les providers externes

Le Wallet ne communique **jamais** directement avec Orange Money, Stripe ou tout autre provider. Pour toute entrée ou sortie d'argent réel, il passe par le Service Paiement. Le Wallet ne gère que la **comptabilité interne**.

7.4 Stratégie d'intégrité

Intégrité des données financières

Transactions atomiques : chaque opération d'écriture (crédit, débit, virement, split) est encapsulée dans une transaction PostgreSQL. Si une seule ligne échoue, tout est annulé (rollback).
Niveau d'isolation : les écritures ledger utilisent `SERIALIZABLE` ou `SELECT ... FOR UPDATE` pour éliminer les race conditions sur les soldes.
Solde caché : le solde est maintenu dans un champ `balance` sur la table `accounts` pour la performance (lecture < 50ms). Ce champ est mis à jour **atomiquement dans la même transaction** que l'écriture ledger. Un cron quotidien vérifie que `balance` $= \sum$ crédits $- \sum$ débits pour chaque compte.
Append-only : le ledger est en insertion seule. Aucun UPDATE, aucun DELETE sur les écritures. Les corrections passent par des écritures inverses (reversal).

7.5 Diagramme d'architecture

`AGT-Market` `MboaMove` `SALMA` `AGT-Bot` `Futures`

```
     |           |          |        |         |
     +-----+-----+-----+----+--------+---------+
           |           |
   Wallet Service (API REST + Workers)
     |         |          |           |
 Auth(JWT)  Users      Redis      PostgreSQL
     |                               (ledger)
 RabbitMQ
   |       |
Paiement  Notification
   |
[providers externes]
```

7.6 Flux financiers (v1.1)

```
=== CASH-IN ===
Provider externe --> [Paiement] --> payment.confirmed
                                        |
                                 [Wallet: DEBIT external, CREDIT wallet_user]

=== ACHAT AVEC ESCROW ===
Client paie --> [Paiement] --> payment.confirmed
                                 |
                             [Wallet: DEBIT external, CREDIT escrow]
                                 |
                         Livraison confirmee
                                 |
                             [Wallet: DEBIT escrow,
                                      CREDIT wallet_vendeur (85%),
                                      CREDIT platform_agt (15%)]

=== CASH-OUT ===
User demande retrait
     |
[Wallet: hold sur wallet_user]
     |
wallet.cashout.requested --> [Paiement] --> provider
     |                                        |
payment.confirmed                      payment.failed
     |                                        |
[Wallet: DEBIT escrow,                [Wallet: release hold,
 CREDIT external]                      fonds restitues]

=== VIREMENT ===
[Wallet: DEBIT wallet_A, CREDIT wallet_B] (atomique)
```

8 Modélisation

8.1 Diagramme de Classes

Le diagramme de classes complet est fourni en annexe au format PlantUML (voir section 12). Il couvre les entités : `Account`, `LedgerTransaction`, `LedgerEntry`, `Hold`, `CashoutRequest` et

`SplitRule`.

8.2 MLD — Modèle Logique de Données

8.2.1 Table `accounts`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
account_type	VARCHAR(20)	NOT NULL	user, organization, platform_system, escrow, external
owner_type	VARCHAR(20)	NOT NULL	user, organization, system
owner_id	UUID	NULL	`users_auth.id` ou `organizations.id`. NULL pour comptes système.
currency	VARCHAR(3)	NOT NULL	Devise du compte (XAF, EUR, USD)
balance	DECIMAL(15,2)	DEFAULT 0	Solde caché (source de vérité = ledger)
hold_amount	DECIMAL(15,2)	DEFAULT 0	Total des holds en cours
status	VARCHAR(20)	DEFAULT 'active'	active, frozen, closed
label	VARCHAR(100)	NULL	Libellé (ex : « Wallet principal », « Escrow Market »)
metadata	JSONB	NULL	Données complémentaires
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

Contrainte unique : `(owner_type, owner_id, account_type, currency)` pour les comptes user/org. Les comptes système ont un `label` unique.

8.2.2 Table `ledger_transactions`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
ledger_reference_id	VARCHAR(30)	UNIQUE, NOT NULL	Référence séquentielle lisible (ex : `LTX-2026-000001`). Générée automatiquement.
idempotency_key	UUID	UNIQUE, NOT NULL	Clé d'idempotence
transaction_type	VARCHAR(30)	NOT NULL	cashin, cashout, transfer, split, hold_capture, hold_release, adjustment, reversal
platform_id	UUID	NOT NULL	Plateforme source
source	VARCHAR(30)	NOT NULL	payment, platform, admin, cron
source_reference_id	UUID	NULL	Référence externe (payment_id, order_id)
description	VARCHAR(255)	NULL	Description lisible
metadata	JSONB	NULL	Données opaques
status	VARCHAR(20)	DEFAULT 'completed'	completed, reversed
created_at	TIMESTAMPTZ	NOT NULL	Date de l'opération

8.2.3 Table `ledger_entries`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
transaction_id	UUID	FK → ledger_transactions	Transaction parent
account_id	UUID	FK → accounts	Compte concerné
direction	VARCHAR(6)	NOT NULL	`debit` ou `credit`
amount	DECIMAL(15,2)	NOT NULL	Montant (toujours positif)
balance_after	DECIMAL(15,2)	NOT NULL	Solde du compte après cette écriture
created_at	TIMESTAMPTZ	NOT NULL	Date

Index : `(account_id, created_at)` pour l'historique par compte.

Contrainte d'équilibre

Une contrainte applicative (vérifiée avant insertion) garantit que pour chaque `ledger_transaction`, la somme des `amount` où `direction = 'debit'` est **strictement égale** à la somme des `amount` où `direction = 'credit'`. Toute violation est rejetée.

8.2.4 Table holds

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
account_id	UUID	FK → accounts	Compte sur lequel le hold est placé
amount	DECIMAL(15,2)	NOT NULL	Montant réservé
status	VARCHAR(20)	DEFAULT 'pending'	pending, captured, released, expired
reason	VARCHAR(50)	NOT NULL	cashout, order, escrow
reference_id	UUID	NULL	Référence externe (cashout_request_id, order_id)
idempotency_key	UUID	UNIQUE, NOT NULL	Idempotence
expires_at	TIMESTAMPTZ	NOT NULL	Expiration du hold
captured_amount	DECIMAL(15,2)	DEFAULT 0	Montant effectivement capturé (partiel possible)
created_at	TIMESTAMPTZ	NOT NULL	Création
resolved_at	TIMESTAMPTZ	NULL	Date de capture/release/expiration

8.2.5 Table cashout_requests

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
account_id	UUID	FK → accounts	Wallet source
amount	DECIMAL(15,2)	NOT NULL	Montant demandé
currency	VARCHAR(3)	NOT NULL	Devise
destination_provider	VARCHAR(30)	NOT NULL	orange_money, mtn_momo, bank_transfer
destination_details	JSONB	NOT NULL	Numéro téléphone ou IBAN (chiffré)
status	VARCHAR(20)	DEFAULT 'pending'	pending, processing, completed, failed, cancelled
hold_id	UUID	FK → holds	Hold associé
payment_tx_id	UUID	NULL	Transaction Paiement associée
idempotency_key	UUID	UNIQUE, NOT NULL	Idempotence
failure_reason	TEXT	NULL	Raison de l'échec
created_at	TIMESTAMPTZ	NOT NULL	Création
completed_at	TIMESTAMPTZ	NULL	Date de complétion

8.2.6 Table split_rules

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant unique
platform_id	UUID	NOT NULL	Plateforme
name	VARCHAR(100)	NOT NULL	Nom de la règle (ex : « market_sale », « ride_completed »)
splits	JSONB	NOT NULL	Définition des parts : `[{"target":"seller", "percent":85}, {"target":"platform", "percent":15}]`
is_active	BOOLEAN	DEFAULT true	Règle active
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	Dernière MAJ

Contrainte unique : `(platform_id, name)`.

9 Contrats d'API

Base URL : `/api/v1`. Tous les endpoints retournent du JSON. Les endpoints protégés requièrent `Authorization: Bearer <token>`.

9.1 Health Check

9.1.1 GET /wallet/health

Response 200 :

```
{
  "status": "healthy",
  "database": "ok",
  "redis": "ok",
  "rabbitmq": "ok",
  "ledger_balanced": true,
  "version": "1.1.0"
}
```

9.2 Comptes

9.2.1 GET /wallet/accounts/me

Retourne le wallet de l'utilisateur authentifié. Auth : Bearer token.

Response 200 :

```
{
  "account_id": "acc-uuid",
  "account_type": "user",
  "currency": "XAF",
  "balance": 45000,
  "hold_amount": 5000,
  "available_balance": 40000,
  "status": "active"
}
```

Codes : 200, 401, 404 No wallet (création automatique possible).

9.2.2 GET /wallet/accounts/{accountId}

Détail d'un compte. Auth : admin ou propriétaire. Codes : 200, 403, 404.

9.2.3 POST /wallet/accounts/{accountId}/freeze

Gel d'un wallet. Auth : admin. Codes : 200, 403, 404.

9.2.4 POST /wallet/accounts/{accountId}/unfreeze

Dégel. Auth : admin. Codes : 200, 403, 404.

9.3 Opérations Ledger

9.3.1 POST /wallet/credit

Crédit d'un compte. Auth : S2S ou admin.

Request body :

```
{
  "account_id": "acc-uuid",
  "amount": 10000,
  "currency": "XAF",
  "platform_id": "uuid",
  "source": "payment",
  "source_reference_id": "payment-tx-uuid",
  "idempotency_key": "unique-uuid",
  "description": "Cash-in via Orange Money"
}
```

Response 201 :

```
{
  "transaction_id": "ltx-uuid",
  "account_id": "acc-uuid",
  "amount": 10000,
  "new_balance": 55000,
  "message": "Credit completed"
}
```

Codes : 201, 200 Idempotent hit, 400, 401, 404 Account not found, 409 Account frozen.

9.3.2 POST /wallet/debit

Débit d'un compte. Auth : S2S ou admin.

Request body :

```
{
  "account_id": "acc-uuid",
  "amount": 5000,
  "currency": "XAF",
  "platform_id": "uuid",
  "source": "platform",
  "source_reference_id": "order-uuid",
  "idempotency_key": "unique-uuid",
```

```
  "description": "Commission AGT"
}
```

Codes : 201, 200 Idempotent, 400, 403 Insufficient balance, 404, 409 Account frozen.

9.3.3 POST /wallet/transfer

Virement entre deux comptes. Auth : S2S, admin, ou propriétaire du compte source.

Request body :

```
{
  "from_account_id": "acc-uuid-1",
  "to_account_id": "acc-uuid-2",
  "amount": 3000,
  "currency": "XAF",
  "platform_id": "uuid",
  "idempotency_key": "unique-uuid",
  "description": "Paiement fournisseur"
}
```

Codes : 201, 200 Idempotent, 400, 403 Insufficient balance, 404, 409 Account frozen.

9.3.4 POST /wallet/split

Partage d'un montant selon une règle de split. Auth : S2S.

Request body :

```
{
  "source_account_id": "escrow-uuid",
  "amount": 10000,
  "currency": "XAF",
  "platform_id": "uuid",
  "split_rule_id": "rule-uuid",
  "targets": {
    "seller": "acc-uuid-vendeur",
    "platform": "acc-uuid-agt"
  },
  "idempotency_key": "unique-uuid",
  "source_reference_id": "order-uuid"
}
```

Response 201 :

```
{
  "transaction_id": "ltx-uuid",
  "entries": [
    { "account": "escrow", "direction": "debit", "amount": 10000 },
    { "account": "vendeur", "direction": "credit", "amount": 8500 },
    { "account": "agt", "direction": "credit", "amount": 1500 }
  ],
  "message": "Split completed"
}
```

Codes : 201, 200 Idempotent, 400 Split unbalanced, 403, 404.

9.3.5 GET /wallet/accounts/{accountId}/transactions

Historique des mouvements d'un compte. Query params : `?type&from_date&to_date&page&limit`.

Codes : 200, 403, 404.

9.4 Holds

9.4.1 POST /wallet/holds

Création d'un hold. Auth : S2S.

Request body :

```
{
  "account_id": "acc-uuid",
  "amount": 5000,
  "reason": "cashout",
  "reference_id": "cashout-req-uuid",
  "idempotency_key": "unique-uuid",
  "ttl_seconds": 3600
}
```

Codes : 201, 403 Insufficient available balance, 404, 409 Account frozen.

9.4.2 POST /wallet/holds/{holdId}/capture

Capture totale ou partielle. Auth : S2S.

Request body :

```
{
  "amount": 5000
}
```

Codes : 200, 400 Amount exceeds hold, 404, 410 Hold expired.

9.4.3 POST /wallet/holds/{holdId}/release

Libération du hold. Auth : S2S. Codes : 200, 404, 410 Already captured.

9.5 Cash-out

9.5.1 POST /wallet/cashout

Demande de retrait. Auth : Bearer token (propriétaire du wallet).

Request body :

```
{
  "amount": 5000,
  "currency": "XAF",
  "destination_provider": "orange_money",
  "destination_details": {
    "phone_number": "+237600000000"
  },
  "idempotency_key": "unique-uuid"
}
```

Response 201 :

```
{
  "cashout_id": "co-uuid",
  "status": "pending",
  "amount": 5000,
  "hold_id": "hold-uuid",
  "message": "Cashout request created, processing"
}
```

Codes : 201, 400 Below minimum / above daily limit, 403 Insufficient balance, 409 Account frozen.

9.5.2 GET /wallet/cashout/{cashoutId}

Détail d'une demande de retrait. Codes : 200, 404.

9.5.3 GET /wallet/cashout

Liste des demandes de retrait. Query params : `?status&from_date&to_date&page&limit`. Codes : 200, 401.

9.6 Split Rules

9.6.1 POST /wallet/split-rules

Création d'une règle de partage. Auth : admin plateforme.

Request body :

```
{
  "platform_id": "uuid",
  "name": "market_sale",
  "splits": [
    { "target": "seller", "percent": 85 },
    { "target": "platform", "percent": 15 }
  ]
}
```

Codes : 201, 400 Percentages don't sum to 100, 409 Name exists.

9.6.2 GET /wallet/split-rules

Query params : `?platform_id`. Codes : 200.

9.6.3 PUT /wallet/split-rules/{ruleId}

Modification. Codes : 200, 400. *Note : modifier une règle n'affecte pas les transactions déjà exécutées.*

9.7 Administration

9.7.1 GET /wallet/admin/stats

Query params : `?platform_id`. Auth : admin.

Codes : 200, 403.

9.7.2 POST /wallet/admin/audit-ledger

Lance une vérification d'équilibre du ledger. Auth : admin global.

Response 200 :

```
{
  "balanced": true,
  "total_debits": 185000000,
  "total_credits": 185000000,
  "accounts_checked": 12450,
  "anomalies": 0
}
```

Codes : 200, 403.

9.7.3 POST /wallet/admin/adjustment

Crédit ou débit correctif. Auth : admin global. Justification obligatoire.

Request body :

```
{
  "account_id": "acc-uuid",
  "amount": 500,
  "direction": "credit",
  "reason": "Compensation erreur technique ref#12345",
  "idempotency_key": "unique-uuid"
}
```

Codes : 201, 400, 403.

9.7.4 DELETE /wallet/by-user/{userId}

Purge RGPD. Auth : S2S. Anonymise `owner_id` sur les comptes et les données personnelles dans `cashout_requests`. Les écritures du ledger sont conservées (obligation comptable).

Codes : 200, 401, 404.

10 Contrats Inter-Services

10.1 Wallet → Auth (JWT / S2S)

Paramètre	Valeur
Endpoint JWT	`GET /auth/verify-token`. Cache Redis TTL 30s.
Endpoint S2S	`POST /auth/s2s/introspect`

10.2 Paiement → Wallet (événements RabbitMQ entrants)

Événement consommé	Action Wallet
`payment.confirmed` où `source = wallet_cashin`	Créditer le wallet utilisateur. Écriture : DEBIT `external` → CREDIT `wallet_user`.
`payment.confirmed` où `source = wallet_cashout`	Cash-out confirmé. Capturer le hold, libérer l'escrow : DEBIT `escrow` → CREDIT `external`.
`payment.failed` où `source = wallet_cashout`	Cash-out échoué. Libérer le hold (fonds retournés au wallet user).
`payment.confirmed` où `source = subscription`	Pas consommé par Wallet (consommé par Abonnement).

10.3 Wallet → Paiement (cash-out)

Événement publié	Payload
`wallet.cashout.requested`	`event_id`, `idempotency_key`, `cashout_request_id`, `amount`, `currency`, `destination_provider`, `destination_details`, `user_id`

10.4 Wallet → Notification (alertes)

Événement	Notification déclenchée
Crédit reçu	Push/email : « Vous avez reçu X FCFA »
Cash-out confirmé	Push/email : « Retrait de X FCFA effectué »
Cash-out échoué	Push : « Échec du retrait, fonds restitués »
Hold expiré	Push : « Réservation de X FCFA libérée »
Solde faible (< seuil)	Push : « Votre solde est faible »

10.5 Wallet — Événements sortants génériques

Événement publié	Payload
`wallet.credited`	`event_id`, `account_id`, `amount`, `currency`, `platform_id`, `source`, `new_balance`
`wallet.debited`	`event_id`, `account_id`, `amount`, `currency`, `platform_id`, `source`, `new_balance`
`wallet.cashout.completed`	`event_id`, `cashout_request_id`, `amount`, `destination_provider`
`wallet.cashout.failed`	`event_id`, `cashout_request_id`, `failure_reason`
`wallet.hold.expired`	`event_id`, `hold_id`, `account_id`, `amount`

10.6 Users → Wallet (Purge RGPD)

Paramètre	Valeur
Endpoint	`DELETE /api/v1/wallet/by-user/{userId}`
Auth	Token S2S
Données anonymisées	`owner_id` → NULL, `destination_details` → NULL. Écritures ledger conservées.
Séquence	Avant Auth purge

11 Matrice d'Autorisation

Endpoint	JWT	S2S	Admin Plat.	Admin Global
GET /wallet/accounts/me	✓			
GET /wallet/accounts/{id}	✓	✓	✓	✓
POST /wallet/accounts/{id}/freeze				✓
POST /wallet/accounts/{id}/unfreeze				✓
POST /wallet/credit		✓		✓
POST /wallet/debit		✓		✓
POST /wallet/transfer	✓	✓		✓
POST /wallet/split		✓		✓
GET /wallet/accounts/{id}/transactions	✓	✓	✓	✓
POST /wallet/holds		✓		✓
POST /wallet/holds/{id}/capture		✓		✓
POST /wallet/holds/{id}/release		✓		✓
POST /wallet/cashout	✓			
GET /wallet/cashout/{id}	✓	✓	✓	✓
GET /wallet/cashout	✓	✓	✓	✓
POST /wallet/split-rules			✓	✓
GET /wallet/split-rules		✓	✓	✓
PUT /wallet/split-rules/{id}			✓	✓
GET /wallet/admin/stats			✓	✓
POST /wallet/admin/audit-ledger				✓
POST /wallet/admin/adjustment				✓
DELETE /wallet/by-user/{userId}		✓		✓

12 Recommandations & Évolutions

- **Intérêts / rendements** : calcul d'intérêts sur les soldes wallet (futur produit financier).
- **Multi-devise par wallet** : un utilisateur avec des sous-comptes en XAF, EUR, USD.
- **Conversion de devises** : taux de change entre sous-comptes.
- **Limites dynamiques** : plafonds de cash-out adaptés au profil de risque (KYC).
- **Notifications enrichies** : relevé mensuel par email.
- **Monitoring** : métriques Prometheus (écritures/s, solde total, holds actifs, anomalies d'équilibre).
- **Versioning API** : préfixe `/api/v1`. Breaking changes → `/api/v2`.

Annexes

Glossaire

Terme	Définition
Ledger	Registre comptable qui enregistre tous les mouvements financiers
Double-entry	Principe comptable : chaque mouvement a un débit et un crédit équilibrés
Hold	Réservation de fonds qui diminue le solde disponible sans débiter
Escrow	Compte de transit sécurisant les fonds en attente de confirmation
Cash-in	Entrée d'argent dans l'écosystème (depuis un provider externe)
Cash-out	Sortie d'argent de l'écosystème (vers un provider externe)
Split	Répartition d'un montant entre plusieurs comptes selon des pourcentages
Reversal	Écriture inverse annulant une écriture précédente
Balance	Solde d'un compte = $\sum$ crédits − $\sum$ débits
Available balance	Solde disponible = balance − holds pending
S2S	Service-to-Service
DLQ	Dead Letter Queue

Code PlantUML — Diagramme de Classes

Le code PlantUML ci-dessous correspond au diagramme de classes complet du service Wallet v1.1. Il peut être rendu sur `plantuml.com` ou tout outil compatible.

```
@startuml Wallet Service - Class Diagram v1.1

skinparam classAttributeIconSize 0
skinparam linetype ortho

entity "accounts" as acc {
  * id : UUID <<PK>>
  --
  * account_type : VARCHAR(20)
  * owner_type : VARCHAR(20)
  owner_id : UUID
  * currency : VARCHAR(3)
  balance : DECIMAL(15,2) = 0
  hold_amount : DECIMAL(15,2) = 0
  status : VARCHAR(20) = 'active'
  label : VARCHAR(100)
  metadata : JSONB
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  <<UQ>> (owner_type, owner_id, account_type, currency)
}

entity "ledger_transactions" as ltx {
  * id : UUID <<PK>>
  --
  * ledger_reference_id : VARCHAR(30) <<UQ>>
  * idempotency_key : UUID <<UQ>>
  * transaction_type : VARCHAR(30)
  * platform_id : UUID
  * source : VARCHAR(30)
  source_reference_id : UUID
  description : VARCHAR(255)
  metadata : JSONB
  status : VARCHAR(20) = 'completed'
  * created_at : TIMESTAMPTZ
}

entity "ledger_entries" as entries {
  * id : UUID <<PK>>
  --
  * transaction_id : UUID <<FK>>
  * account_id : UUID <<FK>>
```

```
  * direction : VARCHAR(6)
  * amount : DECIMAL(15,2)
  * balance_after : DECIMAL(15,2)
  * created_at : TIMESTAMPTZ
}

entity "holds" as holds {
  * id : UUID <<PK>>
  --
  * account_id : UUID <<FK>>
  * amount : DECIMAL(15,2)
  status : VARCHAR(20) = 'pending'
  * reason : VARCHAR(50)
  reference_id : UUID
  * idempotency_key : UUID <<UQ>>
  * expires_at : TIMESTAMPTZ
  captured_amount : DECIMAL(15,2) = 0
  * created_at : TIMESTAMPTZ
  resolved_at : TIMESTAMPTZ
}

entity "cashout_requests" as cashouts {
  * id : UUID <<PK>>
  --
  * account_id : UUID <<FK>>
  * amount : DECIMAL(15,2)
  * currency : VARCHAR(3)
  * destination_provider : VARCHAR(30)
  * destination_details : JSONB
  status : VARCHAR(20) = 'pending'
  hold_id : UUID <<FK>>
  payment_tx_id : UUID
  * idempotency_key : UUID <<UQ>>
  failure_reason : TEXT
  * created_at : TIMESTAMPTZ
  completed_at : TIMESTAMPTZ
}

entity "split_rules" as splits {
  * id : UUID <<PK>>
  --
  * platform_id : UUID
  * name : VARCHAR(100)
  * splits : JSONB
  is_active : BOOLEAN = true
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
  ..
  <<UQ>> (platform_id, name)
}

ltx ||--o{ entries     : "2..* entries (balanced)"
acc ||--o{ entries     : "0..* entries"
acc ||--o{ holds       : "0..* holds"
acc ||--o{ cashouts    : "0..* cashout requests"
holds ||--o| cashouts  : "0..1 cashout"

@enduml
```